

AWS S3

(Simple Storage Service)

1. What is S3?

- **Object storage** - store & retrieve any amount of data, from anywhere
- Uses: backup/restore, static website, app files, data lake, logs, media, archive



2. Key terms

- **Bucket** = container for objects. Name must be **globally unique**
- **Object** = the data stored. Has: data, key, metadata, tags (optional), version ID (if versioning on)
- **Key** = full name of object in bucket, e.g. documents/aws/notes.pdf
- No real folders in S3! "Folder" = just a **prefix** in the key

3. Object vs Block vs File

Type	Service	Use
Object	S3	files, backups, media
Block	EBS	EC2 disks
File	EFS	shared filesystem

Remember: S3 = Objects, EBS = Blocks, EFS = Files

Storage Classes

- **Standard** - frequently accessed, general purpose, high availability + durability

- **Intelligent-Tiering** - auto moves objects between tiers; use when access pattern unknown / changing
- **Standard-IA** - infrequent access, cheaper storage, but retrieval charge + minimum storage duration
- **One Zone-IA** - like IA but stored in ONE AZ, cheaper. Only for data you can recreate
- **Glacier Instant Retrieval** - archive, still fast access
- **Glacier Flexible Retrieval** - cheaper archive, slower retrieval
- **Glacier Deep Archive** - cheapest, long-term archive, slowest



frequent -> unknown -> infrequent -> archive -> very long term

Durability vs Availability

- **Durability** = chance data stays intact. S3 = **11 nines** (99.999999999%)
- **Availability** = how often data is accessible. Depends on storage class

Durability is NOT Availability (common interview Q)

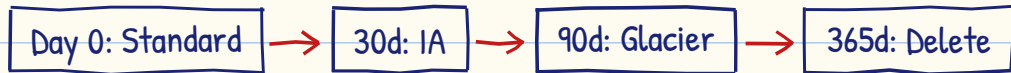
Versioning

- Keeps **multiple versions** of same object (v1, v2, v3...)
- Protects from accidental delete / overwrite; can restore old version
- Delete with versioning on -> S3 adds a **delete marker**, old version NOT removed

Lifecycle Rules

- Auto manage objects as they age -> cut cost
- Two actions: **Transition** (move class) and **Expiration** (delete)

Example:



Encryption (at rest)

Server-side

- **SSE-S3** - S3 manages keys. Simple default
- **SSE-KMS** - AWS KMS keys. More control + audit trail
- **SSE-C** - customer gives key with each request; AWS does encrypt/decrypt

Client-side

- App encrypts data before upload



Security

- **IAM policy** - attached to user / group / role. Says what that identity can do
- **Bucket policy** - resource-based, attached to the bucket itself
- **Block Public Access** - stops accidental public exposure. Keep ON unless public is deliberate
- **ACL** - old method. AWS recommends IAM + bucket policy instead

- o Modern buckets: **Bucket owner enforced** -> ACLs disabled, bucket owner owns all objects

IAM policy = attached to WHO (identity). Bucket policy = attached to WHAT (bucket).

Static Website + CloudFront

- S3 can host static sites: HTML, CSS, JS, images
- Production setup: put **CloudFront** (CDN) in front of S3



- CloudFront gives: caching, low latency, global delivery, less load on S3, HTTPS
- Private bucket origin -> use **Origin Access Control (OAC)**

Presigned URL

- **Temporary** link to access one private object (download or upload)
- Bucket stays private. URL has an **expiry time**



Multipart Upload

- Big file split into parts, uploaded in parallel
- Benefits: faster, parallel, retry only failed part

Replication

- Auto copy objects to another bucket
- **SRR** = Same-Region Replication
- **CRR** = Cross-Region Replication
- Used for: disaster recovery, compliance, data locality

Versioning must be ON in source AND destination bucket

Event Notifications

- S3 raises event on: object uploaded / deleted / restored
- Targets: **Lambda**, **SQS**, **SNS**, **EventBridge**

Example:



Object Lock

- Blocks delete / overwrite for a set retention period -> **WORM** (Write Once Read Many)
- Modes: **Governance** and **Compliance** . Also **legal hold**
- Used for compliance / regulations

Logging & Monitoring

- **CloudTrail** - API activity, auditing
- **CloudWatch** - metrics, monitoring
- **Server access logging** - detailed request records

Other Features

- **S3 Select** - fetch only needed part of an object (e.g. rows from big CSV), not whole file
- **Transfer Acceleration** - faster upload/download via nearest edge location. For users far from bucket Region
- **Batch Operations** - action on millions of objects (copy, restore, tag, change ACL, invoke Lambda)
- **Inventory** - scheduled report of objects: key, size, class, encryption, version
- **Glacier restore** - archived objects not instantly available; request a restore first, then temporarily accessible

CLI Commands

`aws s3 ls`

list buckets

`aws s3 ls s3://my-bucket`

list objects in bucket

`aws s3 cp file.txt s3://my-bucket/`

upload file

`aws s3 cp s3://my-bucket/file.txt .`

download file

`aws s3 sync ./folder s3://my-bucket/folder`

sync local folder with S3

ARN (for IAM policies)

Bucket : `arn:aws:s3:::bucket-name`

Object : `arn:aws:s3:::bucket-name/object-key`

Object ARN = bucket ARN + / + key

Performance

- Scales automatically to very high request rates. No need to provision capacity
- Large objects -> multipart upload
- Global frequent content -> CloudFront
- Far-away uploaders -> Transfer Acceleration

Interview Quick Answers

- **S3?** object storage to store/retrieve data at scale
- **S3 vs EBS?** object vs block
- **Bucket?** container of objects. **Object?** data identified by key
- **Durability?** 11 nines
- **Versioning?** multiple versions, recover from delete/overwrite
- **Lifecycle?** auto transition / delete by rules
- **Presigned URL?** temporary access to private object
- **CRR vs SRR?** cross-region vs same-region
- **Why CloudFront?** caching, low latency, global, secure delivery
- **Multipart?** large file uploaded in parallel parts
- **Stop public access?** Block Public Access + correct IAM/bucket policies

Big Picture

S3 = Object storage + massive scale + high durability + storage classes + strong access control + lifecycle / replication / automation

Om_chaudhari